Edit

Create a Class from a Prompt Using GitHub Copilot

Challenge Overview

Understand the scenario

You are a Developer at Hexelo using GitHub Copilot in VS Code to generate a Python class from a natural-language prompt.

In this Challenge Lab, you will create a Class from a prompt by using GitHub Copilot. First, you will set up your environment and create sample data. Next, you will use GitHub Copilot to generate a working class. Finally, you will enhance the class with validation and new features.

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

Navigating the Challenge Lab

Unable to process content from https://raw.githubusercontent.com/LODSContent/Challenge-V3-Framework/main/Templates/Environments/None.md

▶ Quick tips for navigating the Challenge Lab instructions.

New to Challenge Labs? Click here to learn more.

Prepare the workspace

Hints Enabled

No Yes

⚠ You must complete the **Set Up Git and Connect to GitHub** setup lab before starting any challenge in this series. The set-up lab should only be completed once.

- Sign in to the virtual machine as **LabUser** by using T Passw0rd! as the password.
- Enter your GitHub Username, Email Address, and Password into the following associated textboxes:

  GitHub Username

  GitHub Email Address

  GitHub Password

- Open **Microsoft Edge**, go to T https://github.com, and then sign in as T <GithubUN> by using T <GithubPW> as the password.

Expand this hint for guidance on signing in to a GitHub account.

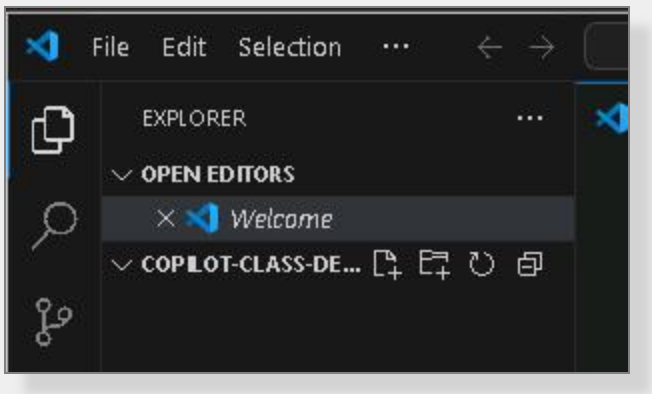
- On the Windows taskbar, in Search, enter **Edge**, and then select **Microsoft Edge** to open a new browser.
- In the Address bar, enter T https://github.com, and then press **Enter**.
- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as T <GithubUN> using T <GithubPW> as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

Leave the **GitHub** browser window open.

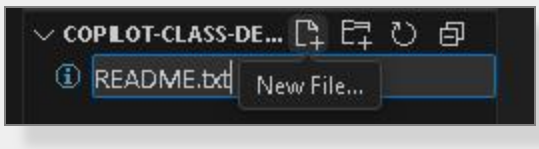
- In T Visual Studio Code, create a project folder named T copilot-class-demo in T D:\LabFiles, and then create a blank T README.txt file to verify folder access.

Expand this hint for guidance on creating a project folder and a blank README.txt file.

- On the Windows taskbar, in Search, enter T Visual Studio Code, and select it to open **VS Code**.
- Close the two *Welcome* tabs.
- In **VS Code**, select **File**, and then select **Open Folder...**.
- Navigate to T D:\LabFiles, select *New folder*, enter T copilot-class-demo to create a folder, double-click on the new folder to open it, and then select **Select Folder**.
- On the *Do you trust the authors of the files in this folder?* dialog, select **Yes, I trust the authors**.
- Confirm the folder appears in the Explorer pane.



- In **VS Code**, select the **New File** icon.



- Name the file T README.txt.

Want to learn more? Review the documentation on opening folders in VS Code.

- Add a sample CSV file to the **copilot-class-demo** folder named T students.csv, and then add the following data:

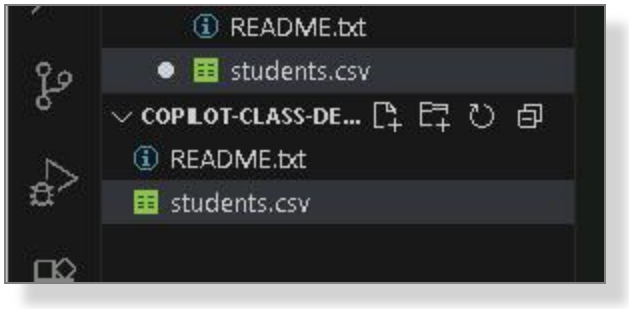
```
name,age,grade
Anna,15,89
Ben,14,92
Cara,15,85
```

Expand this hint for guidance on adding a sample data file.

- In the Explorer, in the **copilot-class-demo** folder, select **New File**, and then name it `students.csv`.
- Copy the following code from above and paste it into the **students.csv** file.
- **Save** the file.

Want to learn more? Review the documentation on working with files in VS Code.

- Confirm that **students.csv** appears in the **copilot-class-demo** folder list.

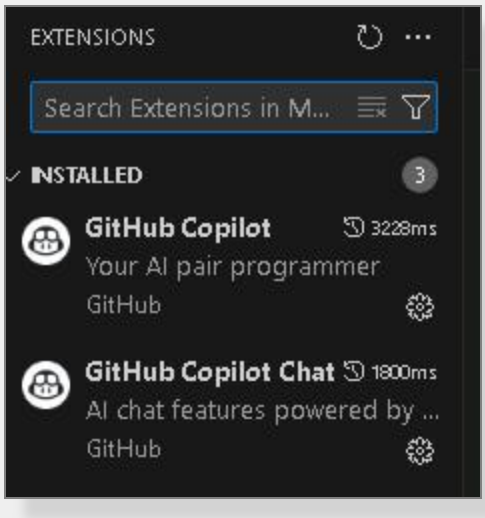


- Verify that GitHub Copilot is installed in ***VS Code***.

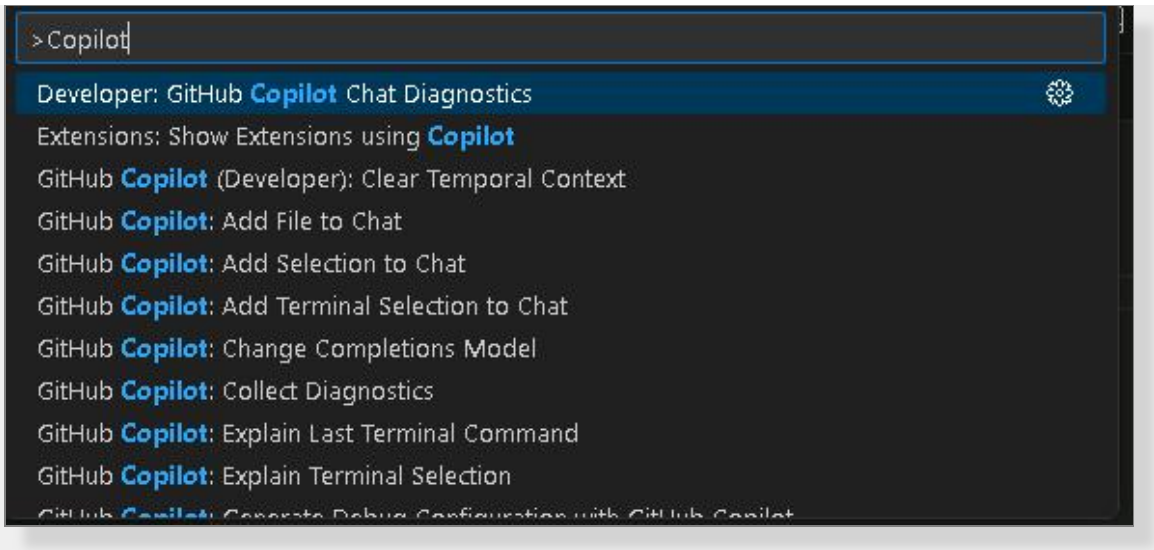
Expand this hint for guidance on verifying GitHub Copilot setup.

- Select the **Extensions** icon in **VS Code**.

- Search for, and if necessary, install:
  - T GitHub Copilot
  - T GitHub Copilot Chat



- Select **Ctrl+Shift+P**, and then enter T Copilot.



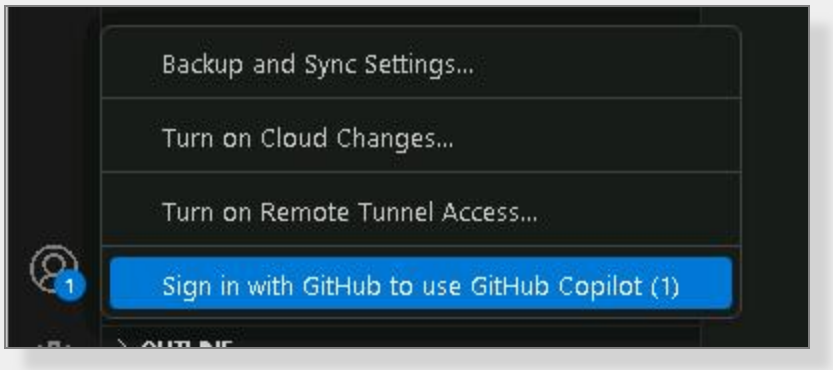
- Confirm commands appear.

Want to learn more? Review the documentation on setting up GitHub Copilot in VS Code.

- Sign in with GitHub to use GitHub Copilot as T <GithubUN> by using T <GithubPW> as the password.

Expand this hint for guidance on signing in to a GitHub account in VS Code.

- In **VS Code**, in the lower left-hand corner, select the Accounts icon, select **Sign in**, and then select **Sign in with GitHub to use GitHub Copilot**.



- On the *Authorize Visual Studio Code* page, select **Continue**, and then select **Open**.

  Note: The following steps may not be needed if you have already signed in.

- On the GitHub home page, in the upper right-hand corner, select **Sign in**.
- On the *Sign in to GitHub* page, in Username or email address, sign in as T <GithubUN> using T <GithubPW> as the password.
- If presented with the *Save password* dialog, select **Never**.
- On the *Device verification* page, enter the **Device Verification Code** sent to the email address used to register the account.

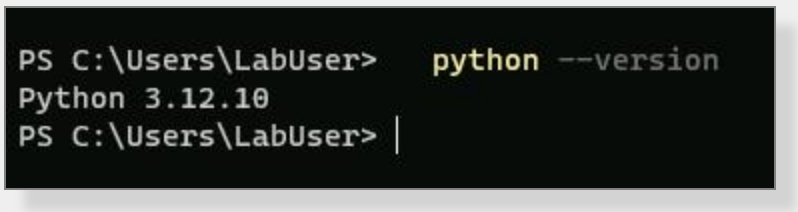
- Open a terminal and confirm Python is installed.

Expand this hint for guidance on confirming Python installation.

- Select the Terminal icon on the taskbar, and then select **New Terminal**.
- Run the following command:

```bash
python --version
```

- Ensure a valid Python version appears.



Want to learn more? Review the documentation on running Python in VS Code.

Check your work

Verify

Generate and test code with Copilot

Hints Enabled

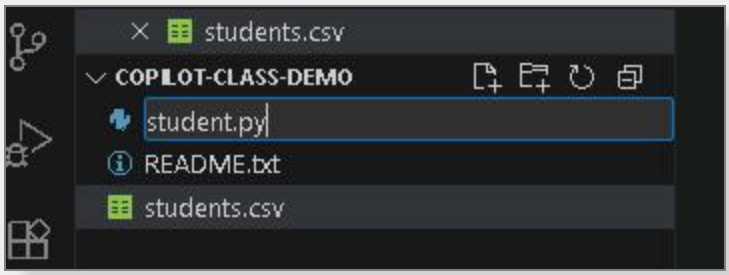
No Yes

- Create a new file named T student.py to seed a descriptive comment block for Copilot by using the following, and then save the file in the **copilot-class-demo** folder:

```python
# Create a class Student:
# - Attributes: name:str, age:int, grade:float
# - Methods:
#     __init__(name, age, grade)
#     __str__ -> "Name (Age): Grade"
#     is_passing() -> bool  # grade >= 70
#     @classmethod load_from_csv(path:str) -> list[Student]
# - Use csv module
# - Handle missing/bad rows safely
# - Include Enter the following hints and docstrings
```

Expand this hint for guidance on seeding a descriptive comment block.

- In the Explorer, select **New File**, and then name it T student.py.



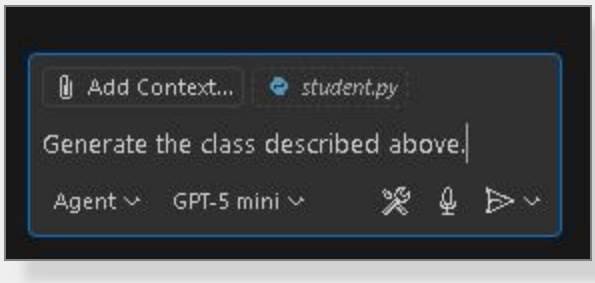
- Copy the comment block provided above and paste it into the new file.
- Move the cursor below the comment.
- **Save** the file.

Want to learn more? Review the documentation on writing effective Copilot prompts.

- Generate the **Student** class using the Copilot Chat with the prompt T Generate the class described above..

Expand this hint for guidance on generating the class using Copilot Inline Chat.

- Select **Ctrl+Shift+I** to open the Copilot Chat window, and then enter the following: T Generate the class described above..



- Select **Send**, and then wait for Copilot's suggestion.
- Ensure it includes the following:
  - __init__
  - __str__
  - is_passing
  - load_from_csv

```
def __init__(self, name: str, age: int, grade:
    """Initialize a Student.

    Args:
        name: student's name
        age: student's age
        grade: student's numeric grade
    """
    self.name = name
    self.age = age
    self.grade = grade

def __str__(self) -> str:
    """Return a readable string: 'Name (Age): G
    return f"{self.name} ({self.age}): {self.gr

def is_passing(self) -> bool:
    """Return True when the student's grade is
    return self.grade >= 70

@classmethod
def load_from_csv(cls, path: str) -> List['Stud
    """Load students from a CSV file.
```

- Select **Keep** to accept the suggestion.

Want to learn more? Review the documentation on using Inline Chat in VS Code.

If missing pieces, run Copilot Chat again with the following prompt:

```
Add required imports and robust error handling.
```

- Add the following code to the **student.py** file, and test the code execution by running the python student.py command in the Terminal.

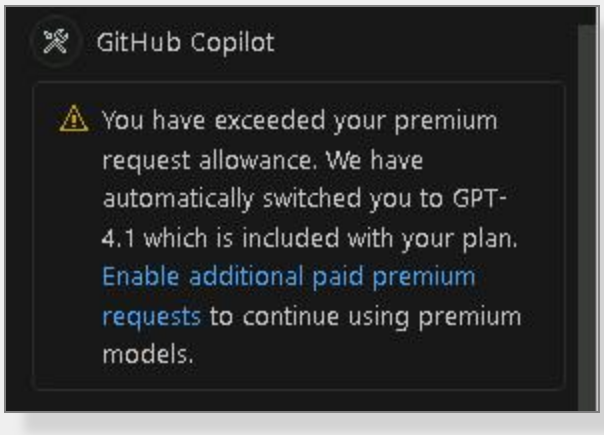
```python
if __name__ == "__main__":
    group = Student.load_from_csv("students.csv")
    for s in group:
        print(s, "Passing:", s.is_passing())
```

Expand this hint for guidance on testing code.

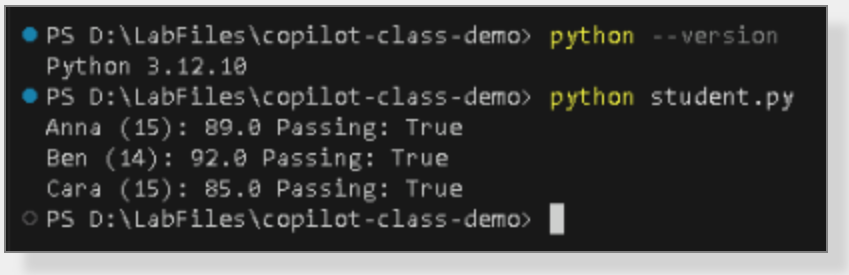
- Copy the code block from above, and append it to the bottom of the **student.py** file.
- In the Terminal, navigate to the **copilot-class-demo** folder, and then run the following command:

```bash
python student.py
```

NOTE: If you receive the following alert, you may ignore it and continue:



- Expected output:



Want to learn more? Review the documentation on running and debugging Python code in VS Code.

Test modifications by adjusting comments and regenerating code.

Want to learn more? Review the documentation on GitHub Copilot.

Check your work

Verify

Enhance code and extend a class with Copilot

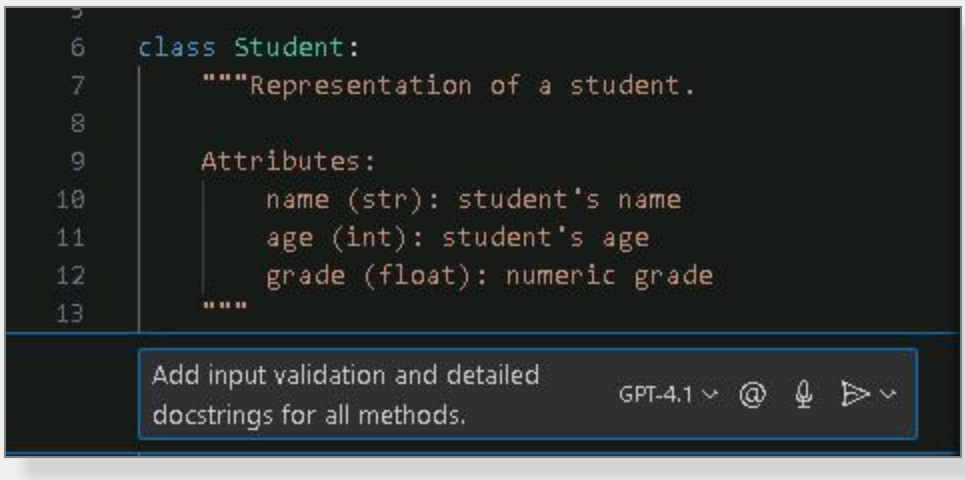
Hints Enabled

No Yes

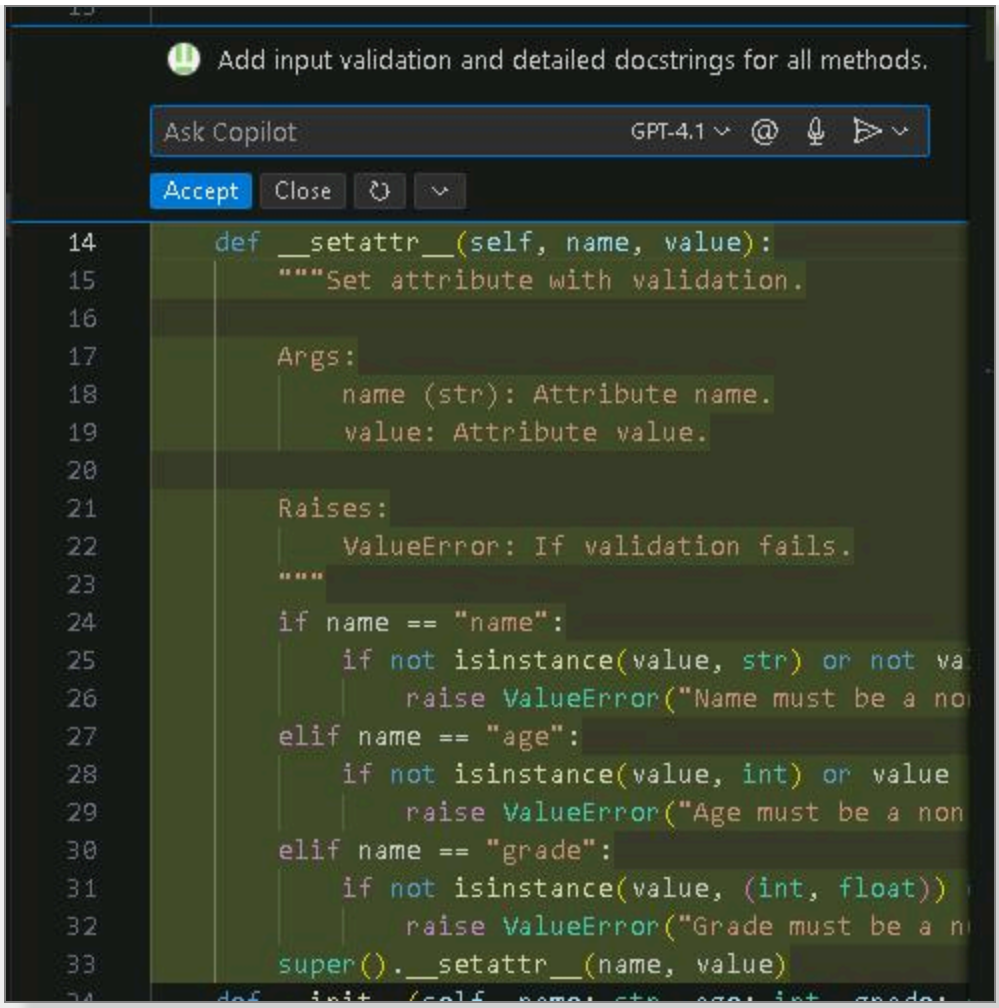
- Add input validation and docstrings by using the prompt T Add input validation and detailed docstrings for all methods. using Copilot Inline Chat.

Expand this hint for guidance on adding input validation and docstrings.

- Select inside the **Student class** definition.
- Open Copilot Inline Chat (**Ctrl+I**), and then enter T Add input validation and detailed docstrings for all methods..



- Select **Send**, and then accept the update and verify changes.



Want to learn more? Review the documentation on Docstring Conventions in Python.

If you end up with two copies of methods, expand this drop-down for help.

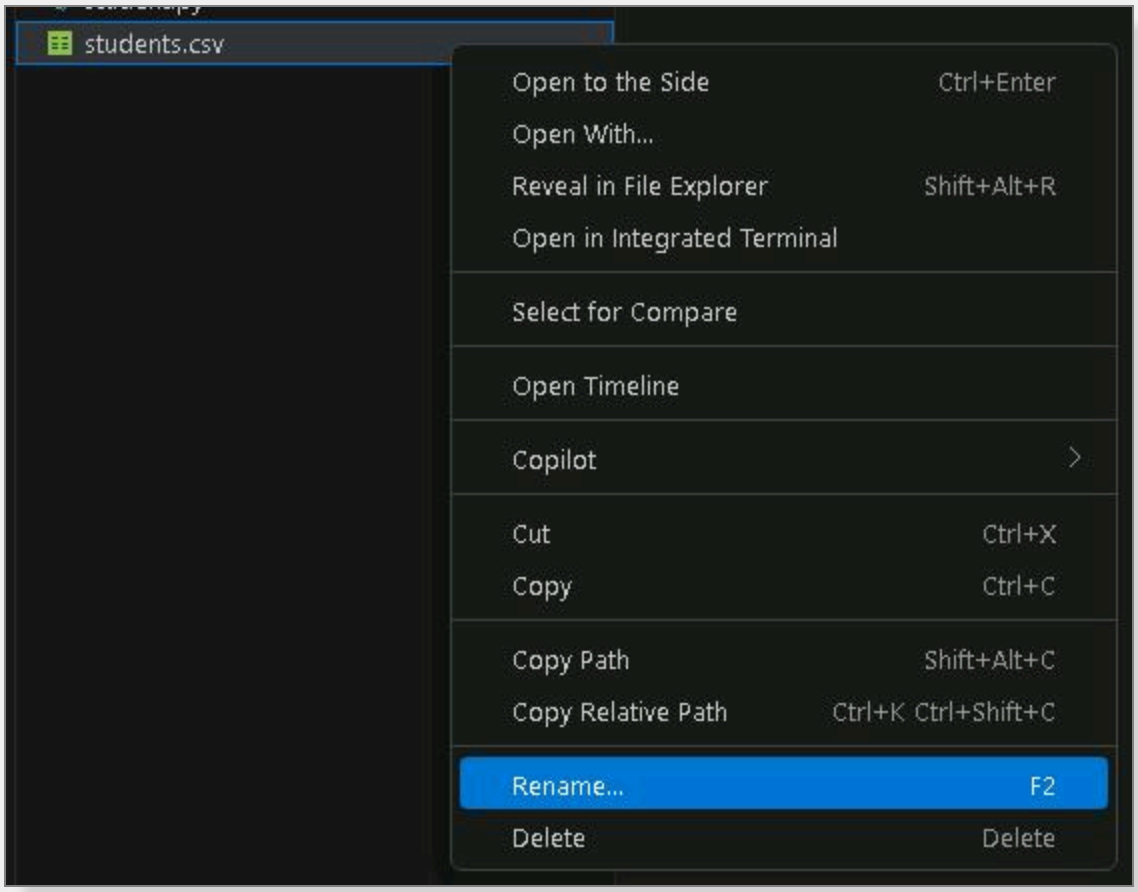
- Enter this prompt into the Copilot Chat window.

```
Remove the older versions of these methods __init__, __str__,
is_passing, and load_from_csv. Keep the ones that are the most
recently generated.
```

- Rename **students.csv** to T students_missing.csv, test the **student.py** script's error handling for missing files, and then restore the original name.

Expand this hint for guidance on testing error handling for missing files.

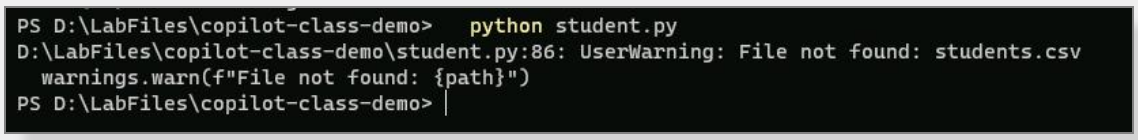
- Rename **students.csv** to T students_missing.csv.



- In the Terminal, run the following command:

```bash
python student.py
```

- Confirm that the script handles missing files gracefully (no crash).



- Rename back to T students.csv.
- Run the following command:

```bash
```

```
python student.py
```

- Confirm that the script runs as expected.

```
PS D:\LabFiles\copilot-class-demo> python student.py
Anna (15): 89.0 Passing: True
Ben (14): 92.0 Passing: True
Cara (15): 85.0 Passing: True
PS D:\LabFiles\copilot-class-demo>
```

Want to learn more? Review the documentation on handling exceptions in Python.

- Add a new T letter_grade() method by first entering a comment at the end of the class (before the test method), and then using a prompt with Copilot Inline Chat. Use the values in the following table for the comment and the prompt.

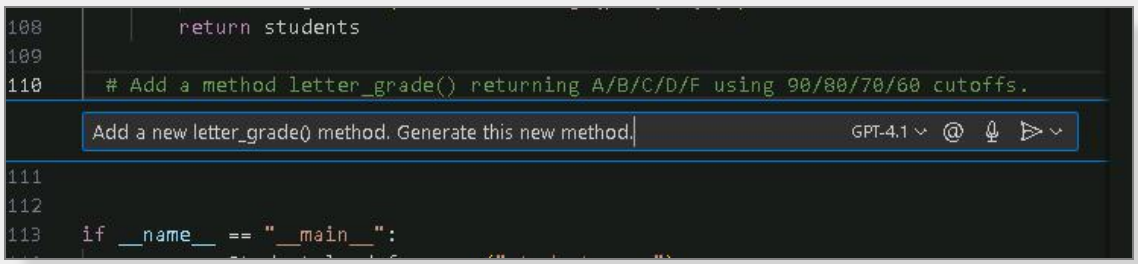
Property	Value
Comment	T # Add a method letter_grade() returning A/B/C/D/F using 90/80/70/60 cutoffs.
Prompt	T Add a new letter_grade() method. Generate this new method.

Expand this hint for guidance on adding a new method.

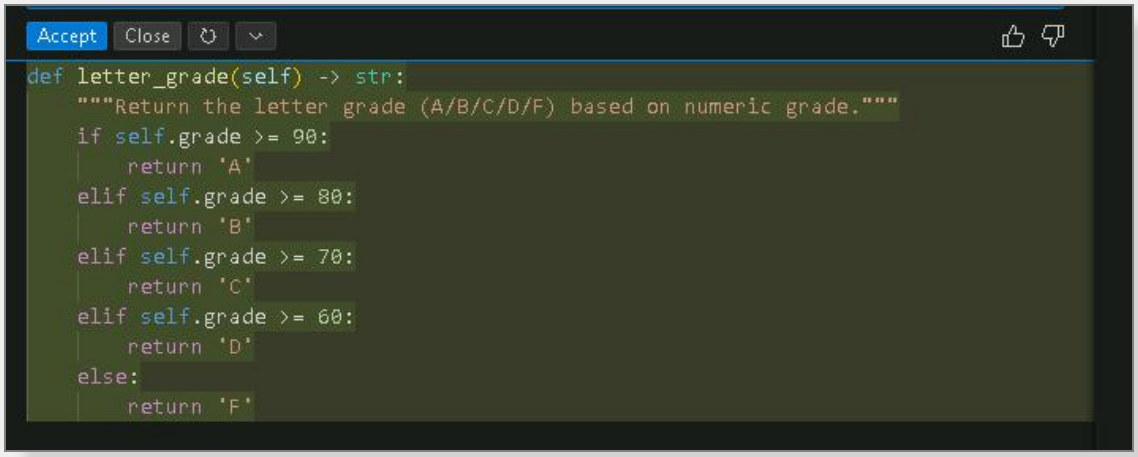
- In **student.py**, add the following comment at the end of the class, before the test method:

```python
# Add a method letter_grade() returning A/B/C/D/F using 90/80/70/60 cuto
```

- In the Copilot Inline Chat (**Ctrl+I**) textbox, enter the following prompt: T Add a new letter_grade() method. Generate this new method. and select **Send**.



- Accept the generated code.



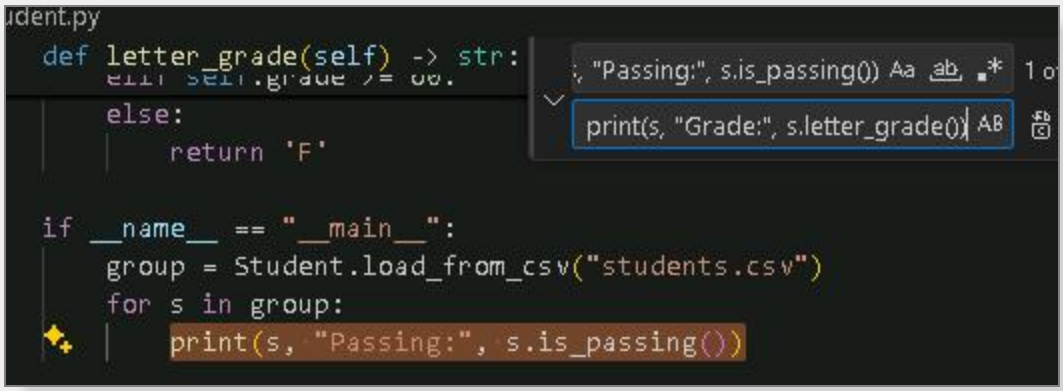
Want to learn more? Review the documentation on using Inline Chat in VS Code.

- Update the test block by using the code below, and then verify the output by using the T python student.py command.

```python
print(s, "Grade:", s.letter_grade())
```

Expand this hint for guidance on updating the test block and verifying output.

- Use the code supplied above to update the test block.



- In the Terminal, run the following command:

```bash
python student.py
```

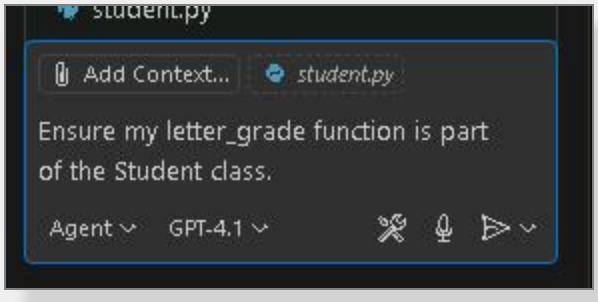
```
PS D:\LabFiles\copilot-class-demo> python student.py
Anna (15): 89.0 Grade: B
Ben (14): 92.0 Grade: A
Cara (15): 85.0 Grade: B
PS D:\LabFiles\copilot-class-demo>
```

If you receive an *AttributeError* error in the Terminal, expand this drop-down menu for help.

Full error message: *AttributeError: 'Student' object has no attribute 'letter_grade'*.

- If you have received the error message above, in the Copilot Chat textbox (**Ctrl+Shift+I**), enter the following prompt:

  T Ensure my letter_grade method is part of the Student class.



- Ensure Copilot has moved the letter_grade method above the test code block.

- Select **Keep**, save the file, and then run the *python* command again:

```bash
python student.py
```

Want to learn more? Review the documentation on:

- Prompt engineering in VS Code
- Prompt examples for chat in VS Code

Check your work

Verify

Summary

Congratulations, you have completed the **Create a Class from a Prompt Using GitHub Copilot** Challenge Lab.

You have accomplished the following:

- Prepared the workspace.
- Generated and tested code with Copilot.
- Enhanced code and extended a class with Copilot.

Ending your lab

To ensure your lab is recorded as complete, select **Submit** or **End** below. Exiting [X] the lab will result in an incomplete status.

⚠ Once you select **Submit** or **End**, you will not be able to return to this Challenge Lab.

Your feedback is important!

As you end your Challenge Lab, please take a few minutes to complete the short survey that will appear in the next window. Alternatively, you may provide your feedback directly to Challenge Labs feedback.

Looking for your next Challenge Lab?

Below are recommended Challenge Labs to try next.

- Set Up Git and Connect to GitHub [Prerequisite Lab]
- Create a Class from a Prompt Using GitHub Copilot [Guided]
- Build Features by Using Copilot in Python [Guided]
- Write Unit Tests Automatically by Using Copilot [Guided]
- Add Comments and Docstrings by Using Copilot [Guided]
- Develop an API Client with GitHub Copilot [Guided]
- Refactor Python Code by Using Copilot [Guided]

- Create Shell Scripts by Using Copilot [Guided]
- Automate CI Workflows by Using GitHub Actions [Guided]
- Document Projects by Using Markdown and Copilot [Guided]
- Refactor and Optimize Code with GitHub Copilot [Guided]
- Build Command-Line Tools by Using Copilot [Guided]
- Utilize Copilot for Python Code Correction [Guided]